

Минобрнауки России
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Доклад по практическому заданию № 1

Сравнительный анализ Jest, Mocha и Vitest по времени выполнения, CPU и ОЗУ

по дисциплине
«Методы тестирования программного обеспечения»

Выполнила: студентка группы 35130903/40002
Фёдорова Ирина Романовна

Руководитель:
старший преподаватель В. А. Пархоменко

Санкт-Петербург
2026

Содержание

1	Введение	2
2	Объект и вопросы исследования	3
3	Математическая постановка проблемы	4
4	Определение времени, CPU и памяти	5
5	Тестируемая задача: палиндромные подстроки	6
6	Алгоритм Манакера и его свойства	7
7	Пример и пошаговое выполнение	8
8	Технология сравнительного эксперимента	9
9	Разработанное программное обеспечение	10
10	Консольный интерфейс приложения	11
11	Форматы данных и эталонные файлы	13
12	Подходы к тестированию корректности	14
13	Проверка измерительного контура	15
14	План эксперимента и нагрузка	16
15	Обработка результатов и воспроизводимость	18
16	Апробация и результаты	19
17	Обсуждение и ограничения	23
18	Инструкция по установке и запуску	24
19	Заключение	26
	Список использованных источников	28

1 Введение

Автоматические тесты выполняются после изменения программы, перед слиянием веток и при сборке выпуска. Поэтому затраты тестовой инфраструктуры многократно повторяются в течение работы над проектом. Даже одинаковая проверка чистой функции может требовать разного времени в зависимости от способа поиска тестов, загрузки модулей и организации исполнителей. Пользователь ожидает быстрый ответ, а сервер непрерывной интеграции имеет ограниченные процессорные и оперативные ресурсы.

В работе рассматривается практическая проблема выбора между Jest, Mocha и Vitest для небольшого проекта JavaScript. Цель состоит в разработке воспроизводимого стенда и сравнении времени полного запуска тестового набора, процессорного времени и наблюдаемого пика оперативной памяти. Проверяемой программой служит авторская реализация алгоритма Манакера. Она выполняет одну и ту же работу во всех трёх средах и принимает строку из консоли, стандартного ввода или JSON-файла.

Предмет исследования ограничен пакетным запуском одного тестового файла в среде Node.js. Работа не исследует браузер, взаимодействие с DOM, сетевые запросы, покрытие, TypeScript и интерактивный режим наблюдения за изменениями. Эти ограничения позволяют точно указать, что означают полученные числа. Результат нельзя автоматически переносить на любой проект с тем же названием фреймворка.

Для достижения цели разработаны загрузчик JSON и CSV, алгоритм поиска палиндромов, общий набор утверждений, адаптеры трёх раннеров, генератор входов и внешний измеритель. Корректность проверяется до измерения производительности. Сырые результаты, журналы и сведения о среде сохраняются вместе с агрегированными значениями. Доклад описывает математическую модель, иллюстративный пример, методы тестирования, реализацию, порядок запуска и интерпретацию эксперимента.

2 Объект и вопросы исследования

Тестовый раннер отвечает за обнаружение и выполнение тестов, обработку ошибок и формирование результата запуска. Библиотека утверждений проверяет соответствие фактического значения ожидаемому. Эти роли можно объединять в одном продукте, однако для эксперимента важно различать их: разное число утверждений способно изменить измеряемую нагрузку.

Jest предоставляет среду выполнения тестов и встроенные средства проверок. В стенде он запускается с `--runInBand`, а преобразования исходников отключены в конфигурации. Это конфигурация исследования, а не утверждение о настройках всех проектов Jest [1, 2].

Mocha используется как последовательный раннер. Проверки выполняет стандартный модуль `node:assert/strict`, вызываемый из общего модуля. Параллельный режим Mocha в эксперимент не включён [3]. Vitest запускается в режиме `run`, с пулом потоков и одним исполнителем. Ограничения `maxWorkers`, `minWorkers` и `fileParallelism` заданы явно [4].

Таблица 1: Экспериментальная конфигурация

Признак	Jest	Mocha	Vitest
Версия	30.0.5	11.7.1	3.2.4
Тестовых файлов	1	1	1
Проверки	общий модуль	общий модуль	общий модуль
Исполнение	последовательно	последовательно	один worker
Среда	Node.js	Node.js	Node.js

Основные вопросы исследования: как изменяется полное время при увеличении входа; совпадает ли лидер по времени с лидером по памяти; насколько велика доля процессорной работы относительно времени ожидания; устойчив ли порядок раннеров между повторениями. Рабочая гипотеза состоит в том, что на малых входах заметны постоянные затраты запуска. Она проверяется наблюдениями, а не принимается как заранее доказанный вывод.

3 Математическая постановка проблемы

Пусть $F = \{\text{Jest}, \text{Mocha}, \text{Vitest}\}$, а $D = \{(s_i, l_i, c_i)\}_{i=1}^k$ представляет входной файл. Здесь s_i является строкой, l_i – ожидаемым самым длинным палиндромом, c_i – числом палиндромных вхождений. Длина $|s_i|$ измеряется в кодовых точках Unicode. Обозначим через $N = \sum_i |s_i|$ суммарный объём строк. При q внутренних повторениях алгоритм обрабатывает qN кодовых точек.

Отдельное наблюдение для раннера f , набора D и повтора r задаётся вектором

$$X_{f,D,r} = (T_{f,D,r}, \widehat{C}_{f,D,r}, \widehat{M}_{f,D,r}, E_{f,D,r}),$$

где T – полное время, $\widehat{C}$ – оценка процессорного времени, $\widehat{M}$ – наблюдаемый пик RSS, E – код завершения. Оценочный характер CPU и памяти отмечен знаком «шляпки». В допустимую выборку входят только наблюдения с успешным завершением и достаточным числом отсчётов.

Задача сравнения формулируется как оценивание медиан вектора затрат:

$$\widetilde{X}_{f,D} = (\text{med}_r T, \text{med}_r \widehat{C}, \text{med}_r \widehat{M}).$$

Для каждой координаты меньшее значение означает меньшие затраты в выбранных условиях. Единственная взвешенная сумма не вводится: время и память имеют разные единицы, а предпочтения пользователя заранее не заданы.

Возможен выбор по ограничению памяти:

$$f^*(D, B) = \arg \min_{f \in F: \widetilde{M}_{f,D} \leq B} \widetilde{T}_{f,D}.$$

Такой выбор является иллюстрацией решения по измеренным оценкам. Для реального жёсткого лимита требуется запас относительно максимального наблюдения и дополнительная проверка на целевой машине. Медиана пиков не гарантирует отсутствие превышения лимита в будущем.

Для сравнения двух раннеров используется отношение $S_{a/b} = \widetilde{T}_a / \widetilde{T}_b$. Если $S_{a/b} > 1$, раннер b быстрее a в данном сценарии. Отношение вычисля-

ется по медианам полного времени и не описывает ускорение самой функции Манакера.

4 Определение времени, CPU и памяти

Полное время измеряется монотонным таймером Python от момента перед созданием процесса до наблюдения его завершения:

$$T = t_{\text{finish}} - t_{\text{start}}.$$

В него включены запуск Node.js, загрузка раннера, поиск тестов, импорт файла, утверждения, завершение и задержка обнаружения выхода наблюдателем. Установка зависимостей и генерация данных выполняются заранее.

Пусть $\mathcal{P}$ обозначает множество обнаруженных процессов дерева. Для процесса p сохраняется последнее наблюдаемое значение суммы пользовательского и системного CPU-времени $u_p + s_p$. Тогда

$$\hat{C} = \sum_{p \in \mathcal{P}} (u_p + s_p)_{\text{last}}, \quad \hat{U} = 100 \frac{\hat{C}}{T}.$$

В таблицах CPU-время представлено в миллисекундах, а $\hat{U}$ – в процентах одного логического процессора. Например, 200% означает среднюю работу, эквивалентную двум полностью занятым логическим процессорам. Это не процент загрузки всей машины. Чтобы получить нормировку по машине с L логическими процессорами, значение делят на L .

Память определяется как

$$\hat{M} = \max_j \sum_{p \in \mathcal{P}_j} \text{RSS}_p(t_j) / 2^{20}.$$

Единица – MiB, то есть 2^{20} байт. RSS характеризует резидентную память процесса; в Windows соответствующее представление связано с рабочим набором [5]. Сумма RSS может учитывать общие страницы несколько раз, поэто-

му её нельзя отождествлять с уникальной физической памятью приложения. Heap JavaScript составляет лишь часть памяти процесса.

Измеритель задаёт паузу 10 мс между обходами. Сам обход также требует времени, поэтому фактический интервал больше и записывается отдельно. Завершившийся между отсчётами процесс может быть пропущен. Последний CPU-интервал тоже может не попасть в сумму. Таким образом, CPU представляет нижнюю оценку накопленных затрат обнаруженного дерева, а память – максимум наблюдений, который может быть ниже истинного кратковременного пика. В отчёте эта погрешность не скрывается округлением.

5 Тестируемая задача: палиндромные подстроки

Палиндромом называется строка, совпадающая со своим обращением. Для строки s требуется найти палиндромную подстроку максимальной длины и число всех непустых палиндромных вхождений. Вхождения считаются по позициям: в строке `aaa` имеются три односимвольных, два двухсимвольных и одно трёхсимвольное вхождение, всего шесть.

При нескольких максимальных ответах выбирается самый левый. Для `babad` результатом становится `bab`, хотя `aba` также имеет максимальную длину. Пустая строка даёт пустой ответ и нулевой счётчик. Строки обрабатываются с учётом регистра, без нормализации Unicode и без удаления пробелов. Поэтому визуально одинаковые последовательности с разными кодовыми точками могут считаться разными.

Прямой перебор всех подстрок с проверкой обращения прост, но при копировании и сравнении подстрок требует до $O(n^3)$ операций. Он удобен как независимый эталон для коротких входов. Для измеряемой нагрузки используется алгоритм Манакера с линейной асимптотикой [6]. Это позволяет увеличивать вход без превращения проверки корректности в чрезмерно долгий перебор.

Строка преобразуется в последовательность с разделителями. Для объ-

яснения применяется запись

$$\text{abba} \mapsto [L, \#, a, \#, b, \#, b, \#, a, \#, R].$$

В настоящем коде L , R и разделитель являются отдельными объектами `Symbol`. Пользовательские символы, похожие на границы, не могут совпасть с ними. Это устраняет ошибку реализации, в которой обычные символы использовались как специальные границы.

Массив $P[i]$ хранит радиус палиндрома вокруг позиции i в преобразованной последовательности. Правый конец текущего самого далеко достигающего палиндрома обозначается R_b , его центр — C_b . Сведения о симметричной позиции позволяют не сравнивать повторно заведомо равные символы.

6 Алгоритм Манакера и его свойства

Алгоритм 1 Поиск самого длинного палиндрома и числа вхождений

Require: Строка s из кодовых точек

```
1:  $t \leftarrow [L, \#, s_0, \#, \dots, s_{n-1}, \#, R]$ 
2:  $P \leftarrow 0; C_b \leftarrow 0; R_b \leftarrow 0; b \leftarrow 0; j \leftarrow 0; z \leftarrow 0$ 
3: for  $i = 1, \dots, |t| - 2$  do
4:   if  $i < R_b$  then
5:      $P[i] \leftarrow \min(R_b - i, P[2C_b - i])$ 
6:   end if
7:   while  $t[i - P[i] - 1] = t[i + P[i] + 1]$  do
8:      $P[i] \leftarrow P[i] + 1$ 
9:   end while
10:  if  $i + P[i] > R_b$  then
11:     $C_b \leftarrow i; R_b \leftarrow i + P[i]$ 
12:  end if
13:  if  $P[i] > b$  then
14:     $b \leftarrow P[i]; j \leftarrow i$ 
15:  end if
16:   $z \leftarrow z + \lfloor (P[i] + 1) / 2 \rfloor$ 
17: end for
18:  $a \leftarrow \lfloor (j - b) / 2 \rfloor$ 
19: return  $(s[a : a + b], z)$ 
```

Границы L и R различны и не встречаются внутри строки. Поэтому расширение остановится прежде, чем оба сравниваемых значения окажутся за пре-

делами массива. Каждое успешное расширение за текущую правую границу увеличивает её. Граница не уменьшается и проходит не больше линейного числа позиций; остальная работа на позицию постоянна. Отсюда получается $O(n)$ времени и $O(n)$ дополнительной памяти.

Самый длинный ответ извлекается по радиусу и центру. Счётчик суммирует число палиндромов, соответствующих каждому центру. Строгое сравнение $P[i] > b$ сохраняет более ранний ответ при равенстве. В коде создаётся массив кодовых точек через `Array.from`, и извлечение результата выполняется из того же массива: смещение индексов UTF-16 и кодовых точек исключено.

7 Пример и пошаговое выполнение

Авторский пример `abba` мал и одновременно содержит чётный палиндром. В преобразованном массиве индексы от 0 до 10 соответствуют последовательности $L, \#, a, \#, b, \#, b, \#, a, \#, R$. Начальные радиусы равны нулю. Состояния после обработки каждой внутренней позиции приведены в таблице.

Таблица 2: Трассировка алгоритма для `abba`

i	$P[i]$	C_b	R_b	Вклад в z	Действие
1	0	1	1	0	разделитель
2	1	2	3	1	символ <code>a</code>
3	0	2	3	0	нет расширения
4	1	4	5	1	символ <code>b</code>
5	4	5	9	2	<code>bb</code> и <code>abba</code>
6	1	5	9	1	зеркальный радиус
7	0	5	9	0	зеркальный радиус
8	1	5	9	1	радиус у границы
9	0	5	9	0	разделитель

На позиции 5 сначала совпадают два символа `b`, затем разделители, два символа `a` и внешние разделители. Получается радиус 4; следующий шаг

сравнивает разные границы L и R и останавливается. Начало ответа равно $\lfloor (5 - 4)/2 \rfloor = 0$, длина равна 4. Сумма вкладов даёт $1 + 1 + 2 + 1 + 1 = 6$.

Можно проверить ответ независимо, выписав все вхождения: a на позиции 0, b на позиции 1, b на позиции 2, a на позиции 3, bb на позициях 1–2 и $abba$ на позициях 0–3. Совпадение с таблицей проверяет и длину, и число вхождений. Радиусы всего массива равны

$$[0, 0, 1, 0, 1, 4, 1, 0, 1, 0, 0].$$

Для демонстрации агрегации рассмотрим специально учебные, не экспериментальные времена: $[100, 110, 130]$ мс. Медиана равна 110 мс. Если в одном запуске измерены $T = 0,110$ с и $\hat{C} = 0,055$ с, то $\hat{U} = 50\%$. Если суммы RSS в трёх отсчётах равны 40, 52 и 47 MiB, наблюдаемый пик равен 52 MiB. Эти числа показывают вычисления метода и не используются в выводах о Jest, Mocha или Vitest.

8 Технология сравнительного эксперимента

Алгоритм 2 Воспроизводимое сравнение тестовых раннеров

Require: Наборы $\mathcal{D}$, раннеры F , повторы $r_{\max}$, q , seed

- 1: Проверить входные наборы общим модулем утверждений
 - 2: Сохранить версии, конфигурации и хеши файлов
 - 3: **for all** $(D, f) \in \mathcal{D} \times F$ **do**
 - 4: Выполнить один исключаемый прогревочный запуск
 - 5: При ошибке остановить эксперимент
 - 6: **end for**
 - 7: **for** $r = 1, \dots, r_{\max}$ **do**
 - 8: $B \leftarrow$ перестановка $\mathcal{D} \times F$ с заданным seed
 - 9: **for all** $(D, f) \in B$ **do**
 - 10: Создать процесс раннера с входом D и повтором q
 - 11: **while** процесс работает **do**
 - 12: Обнаружить потомков, записать CPU и сумму RSS
 - 13: Проверить тайм-аут; выдержать паузу между отсчётами
 - 14: **end while**
 - 15: Сохранить время, код выхода, журнал и качество отсчётов
 - 16: **end for**
 - 17: **end for**
 - 18: Для успешных наблюдений вычислить медианы и квантили
 - 19: При неуспешных запусках завершить команду с ошибкой
-

Перестановка строится блоками: каждая комбинация набора и раннера появляется ровно один раз в каждом раунде. Это уменьшает систематическое преимущество инструмента, который всегда запускался первым или последним. Seed фиксирует порядок, но не делает продолжительность детерминированной: планировщик и фоновые процессы продолжают влиять на результаты.

Каждый замер создаёт новый процесс. Прогрев относится к файлам и кэшам среды, но не сохраняет ЛТ-состояние между процессами. Внутри одного запуска десять повторений нагрузки позволяют функции исполняться многократно. Поэтому корректное название сценария – повторный пакетный запуск с новым процессом и сохранением кэшей ОС. Термин «полностью холодный запуск» к нему не применяется.

9 Разработанное программное обеспечение

Стенд разделяет измеряемую программу и наблюдатель. JavaScript-модули читают вход, вычисляют палиндромы и выполняют утверждения. Python запускает раннеры как дочерние процессы и наблюдает ресурсы извне. Его собственная память не суммируется с памятью измеряемого дерева, хотя вычислительная работа наблюдателя способна косвенно влиять на планирование.

Таблица 3: Основные компоненты приложения

Файл	Назначение
src/cli.js	Консольный ввод строки, JSON и стандартного потока
src/manacher.js	Чистая функция и безопасное преобразование строки
src/data-loader.js	Разбор и валидация JSON/CSV
src/run-suite.js	Общие утверждения, внутренние повторы
src/benchmark.js	Сравнение с отдельным эталонным выходом
src/demo.js	Авторский пример и печать радиусов
tests/*/manacher.test.js	Тонкие адаптеры трёх раннеров
tests/shared.cjs	Общие функциональные и консольные проверки
generate.py	Масштабируемые входы с независимыми ожиданиями
benchmark.py	Измерение, журналы, сводка и метаданные

Общий модуль выполняет строгое сравнение максимального палиндро-

ма и числа вхождений для каждой строки. Положительный результат одного лишь запуска раннера недостаточен: ошибка в любом ожидании приводит к исключению. Адаптеры вызывают один и тот же модуль, что устраняет различия в логике тестов.

Результаты предыдущих внутренних повторов не накапливаются: сохраняется только последний проход. Иначе потребление памяти отражало бы искусственный массив из копий ответов, а не рабочую память алгоритма и раннера. Код снабжён комментариями в местах, существенных для корректности и методики.

Проект не требует сборщика приложения или компиляции TypeScript. После установки зависимостей исходники JavaScript сразу готовы к запуску. Версии JavaScript-пакетов закреплены в `package.json` и транзитивно в `package-lock.json`. Реальный файл `requirements.txt` фиксирует зависимость наблюдателя от `psutil`.

10 Консольный интерфейс приложения

Приложение решает самостоятельную пользовательскую задачу: находит самый длинный палиндром и число всех палиндромных вхождений. Для обычного расчёта пользователь не должен заранее знать ответ. Поэтому интерфейс расчёта отделён от тестового загрузчика с эталонными ожиданиями.

Основной запуск из каталога `benchmark_app`:

```
npm start -- --text "abba"
npm start -- --file data/input-example.json
node src/cli.js --json '{"input":"abba"}'
node src/cli.js --text "abba" --output-json --radii
```

Последняя команда выводит машиночитаемый JSON с результатами. Имена ключей `longest`, `length`, `count` и `radii` сохраняются стабильными для автоматической обработки. Текстовые сообщения и названия тестов написаны на русском. Служебный вывод самих сторонних раннеров, например `PASS` и `Tests`, формируется их библиотеками.

Для строки `abba` консоль показывает:

```
Самый длинный палиндром: "abba"  
Длина: 4  
Число палиндромных вхождений: 6
```

Одинаковые буквы на разных позициях считаются разными вхождениями. Параметр `--radii` добавляет массив радиусов и помогает сопоставить запуск с пошаговой таблицей алгоритма.

Если запустить `prn start` в интерактивном терминале без аргументов, программа запросит строку. Для пустого ввода достаточно нажать Enter. Параметр `--text` позволяет передать строку, начинающуюся с двух дефисов, не принимая её за параметр. Справка доступна через `--help`.

Пользовательский JSON допускает строку, объект с полем `input`, массив строк или объектов, а также объект с массивом `cases`. Например:

```
{"cases": [  
  {"name": "Чётный палиндром", "input": "abba"},  
  {"name": "Русская строка", "input": "топот"},  
  {"name": "Пустая строка", "input": ""}  
]}
```

Поля ожидаемого ответа не требуются. При передаче тестового файла приложению они не участвуют в расчёте: проверку ожиданий выполняют команды тестов и `verify`. Неправильный тип `input`, пустой пакет, повреждённый JSON, отсутствующий файл или конфликт источников ввода дают русское сообщение в стандартном потоке ошибок и код завершения 1.

Для автоматизации предусмотрен `--stdin`. Совместно с `--json` он читает весь поток как JSON, иначе как исходную строку. Пробелы и завершающие переводы строки сохраняются. Это важно при конвейерном вводе: перевод строки, добавленный оболочкой, является частью задачи. Входной UTF-8 BOM удаляется только при разборе JSON.

Интерфейс обрабатывает Unicode по кодовым точкам без нормализации и без изменения регистра. Поэтому А и а различаются, а буква с отдельным комбинировемым знаком не равна одному предсоставленному символу.

Приложение загружает JSON целиком: память зависит от суммарного объёма пакета, хотя один вызов Манакера использует линейную дополнительную память относительно длины одной строки.

11 Форматы данных и эталонные файлы

Тестовый JSON содержит объект с непустым массивом `cases`. Каждая запись имеет уникальное непустое имя, строку `input`, строку `expectedLongest` и неотрицательное целое `expectedCount`. В JSON пустая строка является допустимым входом. Пример записи:

```
{ "cases": [  
  { "name": "Чётный палиндром", "input": "abba",  
    "expectedLongest": "abba", "expectedCount": 6 }  
]}
```

CSV использует одноимённые столбцы:

```
name,input,expectedLongest,expectedCount  
Чётный палиндром,abba,abba,6  
Пустая строка,,,0
```

Для CSV применяется библиотека `csv-parse`, а не разделение строки по запятой. Кавычки и запятые внутри поля требуют настоящего CSV-разбора [7]. Загрузчик не удаляет пробелы внутри входа. Отсутствующее ожидание, повторяющееся имя или недопустимый счётчик приводят к ошибке до начала измерений.

Файлы содержат сами строки переменной длины. Увеличение размера физического файла означает увеличение числа записей или длины строк; скрытой фиксированной нагрузки нет. Генератор принимает размеры через аргументы, записывает JSON и CSV с одинаковым смыслом и позволяет подготовить новые серии.

Сторонние примеры взяты из задачи LeetCode «Longest Palindromic Substring»: `babad` с ответом `bab` и `cbbd` с ответом `bb` [8]. Допускаемый в источнике альтернативный ответ `aba` разрешается правилом выбора самого левого

максимума. Числа вхождений 7 и 5 подсчитаны автором и не приписываются источнику. Они проверяются независимым перебором.

Эталонный вход хранится в `reference-palindromes.json`, отдельный ожидаемый выход — в `reference-output.json`. Команда `npm run verify` сравнивает структуры целиком. Авторские случаи помещены в `author-palindromes.json` и эквивалентный CSV. Происхождение и границы заимствования записаны в `data/SOURCES.md`.

12 Подходы к тестированию корректности

Первая группа проверок представляет функциональное тестирование по спецификации. Для каждой записи сравниваются оба выходных значения. Это сильнее проверки того, что результат непустой: неправильная подстрока той же длины или ошибочное число вхождений должны приводить к отказу.

Вторая группа использует дифференциальное тестирование. Независимая функция перебирает все подстроки, сравнивает их с обращением и выбирает левый максимум. Она не использует радиусы, зеркальные индексы и преобразование Манакера. Исчерпывающе проверяются все двоичные строки длины от 0 до 9:

$$\sum_{n=0}^9 2^n = 2^{10} - 1 = 1023.$$

Этот диапазон достаточно мал для быстрого запуска и содержит пустой вход, повторы, чётные и нечётные палиндромы, а также равные максимумы.

Третья группа является тестированием граничных и специальных значений. Проверяются Unicode-символы вне BMP, символы, похожие на границы алгоритма, перевод строки, кириллица и неоднозначность ответа. Для Unicode программа определяет символ как кодовую точку, а не графему. Тест с комбинируемым знаком фиксирует именно эту семантику.

Четвёртая группа использует метаморфное свойство:

$$\text{count}(s) = \text{count}(\text{reverse}(s)).$$

Обращение устанавливает взаимно однозначное соответствие между палиндромными вхождениями. При этом строковое значение левого максимума может измениться, поэтому для данной проверки сравнивается счётчик.

Наконец, негативные тесты подтверждают, что неверные входы действительно отклоняются. Проверяются пустой набор, отсутствие обязательных полей, отрицательный счётчик и намеренно ошибочный ожидаемый палиндром. Последний случай существенен: он показывает, что проверка способна обнаружить дефект, а не просто успешно исполняет код без содержательного утверждения.

Полный запуск каждого фреймворка регистрирует 12 одинаковых тестов: один пакетный тест эталона и 11 проверок корректности. Общий модуль содержит сами утверждения, а адаптеры только передают функцию регистрации. Команда `npm test` выполняет эти 11 проверок встроенным раннером Node.js и не заменяет запуск Jest, Mocha и Vitest.

При измерении наблюдатель устанавливает `BENCHMARK_MODE=1`. В этом режиме регистрируется только пакетный тест с выбранным файлом и числом повторов. Проверки консоли запускают дочерние процессы и намеренно исключены из измеряемой нагрузки. Тем самым обычный режим проверяет приложение целиком, а эксперимент сохраняет одинаковую алгоритмическую работу.

Тестирование не доказывает корректность для всех строк. Оно дополняет объяснение инварианта алгоритма и покрывает типичные классы ошибок. Процент покрытия кода в этой работе не измеряется; ему не приписывается неподтверждённое значение.

13 Проверка измерительного контура

Интеграционная проверка загружает одинаковые авторские данные из JSON и CSV и сравнивает итоговые структуры. Это проверяет совместную работу парсера, валидации, алгоритма и утверждений. Среди данных есть пустая строка и поле с запятой и кавычкой. Ошибка упрощённого CSV-разделения проявилась бы несовпадением или исключением.

Измерительный контур допускает успешный результат только при нулевом коде завершения, отсутствии тайм-аута, не менее двух отсчётов и положительных CPU и RSS. Нулевой CPU из-за отсутствия наблюдений нельзя трактовать как идеальную экономичность. При неуспешном прогреве серия останавливается; при неуспешном измеряемом запуске сохраняется журнал, а итоговая команда сообщает об ошибке.

Наблюдатель идентифицирует процесс парой PID и временем создания, сохраняет последний CPU-счётчик завершившихся потомков и суммирует RSS только тех, которые доступны при текущем обходе. Состояние дочерних процессов может меняться между вызовами API. Исчезновение процесса обрабатывается отдельно и не считается дефектом тестируемой функции.

У каждого запуска есть собственный журнал. Это позволяет отличить отказ утверждения от отсутствия зависимостей, неверного пути и тайм-аута. Запись raw.json обновляется после каждого измерения: при прерывании остаются уже завершённые наблюдения. Финальный CSV содержит те же строки, а сводка вычисляется из успешной части выборки.

Для быстрой проверки самого стенда предусмотрен короткий запуск с одним набором и двумя повторами. Его нельзя использовать для содержательного ранжирования из-за малой выборки; он предназначен для проверки установки. Перед полной серией нужно убедиться, что все три адаптера выполняют эталонный тест, а заведомо неверный ожидаемый результат приводит к ненулевому выходу каждого раннера.

14 План эксперимента и нагрузка

Основная серия включает три размера строки: 100, 1000 и 10000 кодовых точек. Каждый файл содержит восемь записей, поровну двух семейств. В одном семействе строка состоит только из а. Во втором чередуются а и b. Их ожидаемые результаты выводятся без запуска исследуемого алгоритма.

Для $s = a^n$ любая подстрока является палиндромом:

$$c(s) = n(n + 1)/2, \quad l(s) = s.$$

Для чередующейся строки палиндромны только подстроки нечётной длины:

$$c(s) = \lfloor (n + 1)^2 / 4 \rfloor.$$

Если n чётно, левый максимальный ответ имеет длину $n - 1$, иначе он совпадает со всей строкой. Такие семейства удобны для больших входов, потому что ожидаемый счётчик вычисляется за постоянное число арифметических операций.

Таблица 4: Объём основной нагрузки

Длина строки	Записей	N	$qN, q = 10$
100	8	800	8000
1000	8	8000	80000
10000	8	80000	800000

На каждую комбинацию размера и раннера запланированы один исключаемый прогрев и семь измерений. Всего это 9 прогревов и 63 наблюдения основной серии. Формат основной серии – JSON. CSV проверяется отдельной серией на том же содержимом, чтобы не смешивать изменение формата с изменением нагрузки.

Входы представляют синтетическую алгоритмическую задачу. Они не являются случайной выборкой пользовательских строк и не отражают обычное распределение модулей в большом приложении. Исследование роста размера строки также не эквивалентно исследованию роста числа тестовых файлов: раннер во всех случаях видит один тест с внутренней пакетной обработкой.

15 Обработка результатов и воспроизводимость

Для каждой комбинации вычисляются медиана, первый и третий квартили. Межквартильный размах равен $IQR = Q_3 - Q_1$. Используется включающий метод квартилей библиотеки Python: для отсортированных m значений положение квантиля p равно $(m - 1)p$ с линейной интерполяцией соседних значений. Это определение указано явно, поскольку разные статистические пакеты могут возвращать немного разные квартили.

Медиана менее чувствительна к единичной задержке фонового процесса, чем среднее. Однако она не устраняет систематическое смещение: если антивирус проверяет файлы только одного раннера или процессор всё время работает на сниженной частоте, медиана сохранит это влияние. Поэтому исходные наблюдения не заменяются одной цифрой.

В `metadata.json` записываются ОС, версия Python и Node.js, версии пакетов, число процессоров, объём памяти, команды, параметры, SHA-256 входов, исходников приложения, тестов, конфигураций и lock-файла. Хеш позволяет проверить, что повторный запуск использует тот же файл. Он не фиксирует состояние файловой системы, температуру процессора или внешнюю нагрузку.

Сводная таблица создаётся программно из `summary.json`, а диаграммы – из тех же данных. Ручное перепечатывание чисел исключено. Журналы каждого запуска остаются доступными для проверки. Единицы в таблицах и подписях согласованы: миллисекунды, CPU-проценты одного логического процессора и MiB.

Семь повторов дают описательную выборку, но недостаточны для универсального статистического утверждения. Проверка значимости, доверительные интервалы и причинный вывод о внутренних механизмах раннеров в работе не заявляются. Наблюдаемая разница относится к сохранённым условиям запуска. Для инженерного решения результаты следует повторить на целевой машине и с характерным тестовым набором.

16 Апробация и результаты

Эксперимент выполнен 16.09.2026: Windows 11, Node.js v24.20.0, Python 3.12.14, psutil 7.0.0. В системе 8 физических ядер и 12 логических процессоров, 15.65 GiB оперативной памяти. Версии раннеров закреплены в конфигурации проекта.

В основной серии успешно завершены 63 из 63 запусков. Прогревы исключены. В каждой строке сводных таблиц семь наблюдений. Медиана CPU-процента вычислена по процентам отдельных запусков и не обязана равняться отношению двух медиан.

Таблица 5: Полное время запуска: медиана и квартили

Длина	Раннер	Медиана, мс	Q_1	Q_3
100	Jest	1015.1	940.4	1115.7
100	Mocha	316.3	310.8	354.4
100	Vitest	1269.3	1236.3	1413.6
1000	Jest	1049.4	1026.5	1083.0
1000	Mocha	369.7	345.7	376.0
1000	Vitest	1310.7	1274.4	1522.4
10000	Jest	1758.0	1665.2	1847.2
10000	Mocha	483.6	439.2	492.7
10000	Vitest	1533.2	1344.3	1630.5

Таблица 6: Оценка накопленного CPU-времени дерева процессов

Длина	Раннер	Медиана, мс	Q_1	Q_3
100	Jest	1281.2	1187.5	1476.6
100	Mocha	312.5	296.9	343.8
100	Vitest	1375.0	1359.4	1625.0
1000	Jest	1375.0	1289.1	1382.8
1000	Mocha	328.1	304.7	359.4
1000	Vitest	1562.5	1460.9	1835.9
10000	Jest	1968.8	1859.4	2070.3
10000	Mocha	468.8	445.3	523.4
10000	Vitest	1656.2	1546.9	1789.1

Таблица 7: Наблюдаемый пик суммы RSS: медиана и квартили

Длина	Раннер	Медиана, MiB	Q_1	Q_3
100	Jest	88.4	88.0	88.5
100	Mocha	54.3	54.1	55.2
100	Vitest	124.5	124.2	127.1
1000	Jest	98.6	98.4	99.0
1000	Mocha	56.9	56.1	57.4
1000	Vitest	131.3	130.9	131.5
10000	Jest	113.4	112.9	114.0
10000	Mocha	70.0	69.3	70.1
10000	Vitest	142.8	137.7	143.3

Таблица 8: Средняя CPU-загрузка в единицах одного логического процессора

Длина	Раннер	Медиана, %	Q_1	Q_3
100	Jest	125.4	124.2	129.8
100	Mocha	96.7	90.3	103.5
100	Vitest	110.3	108.8	112.1
1000	Jest	123.6	121.1	127.7
1000	Mocha	97.2	82.0	97.4
1000	Vitest	119.2	116.2	125.4
10000	Jest	112.4	111.1	114.1
10000	Mocha	96.9	96.1	111.6
10000	Vitest	111.3	108.0	113.3

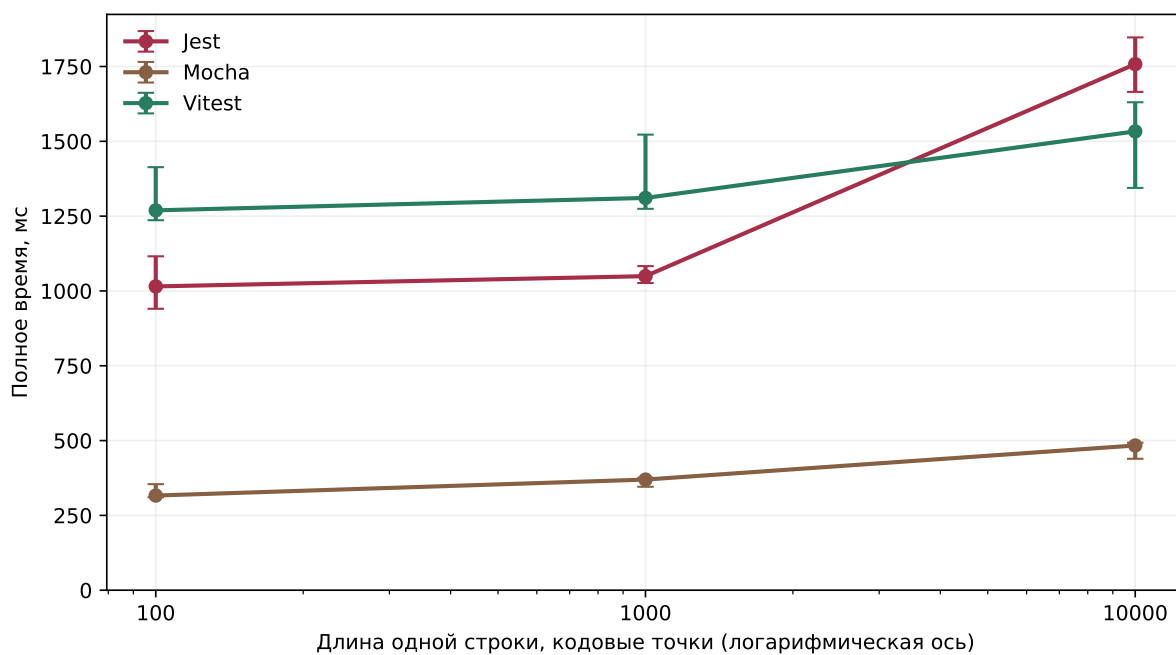


Рис. 1: Полное время, мс. Точки – медианы, отрезки – Q_1 – Q_3 .

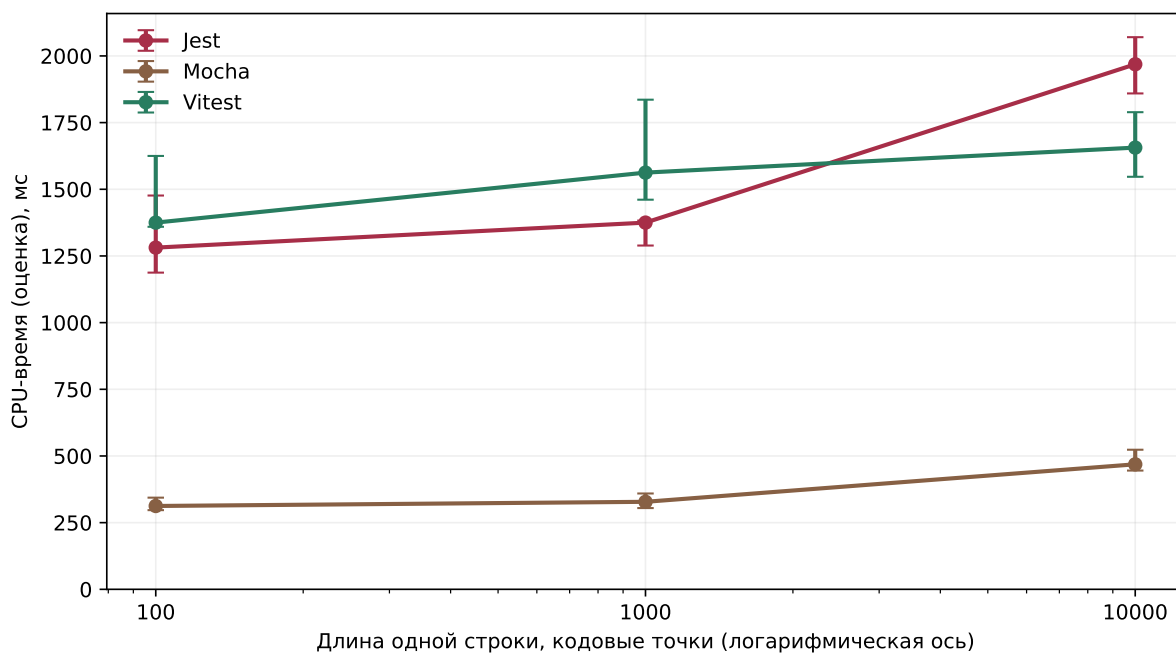


Рис. 2: СРВ-время (оценка), мс. Точки – медианы, отрезки – Q_1 – Q_3 .

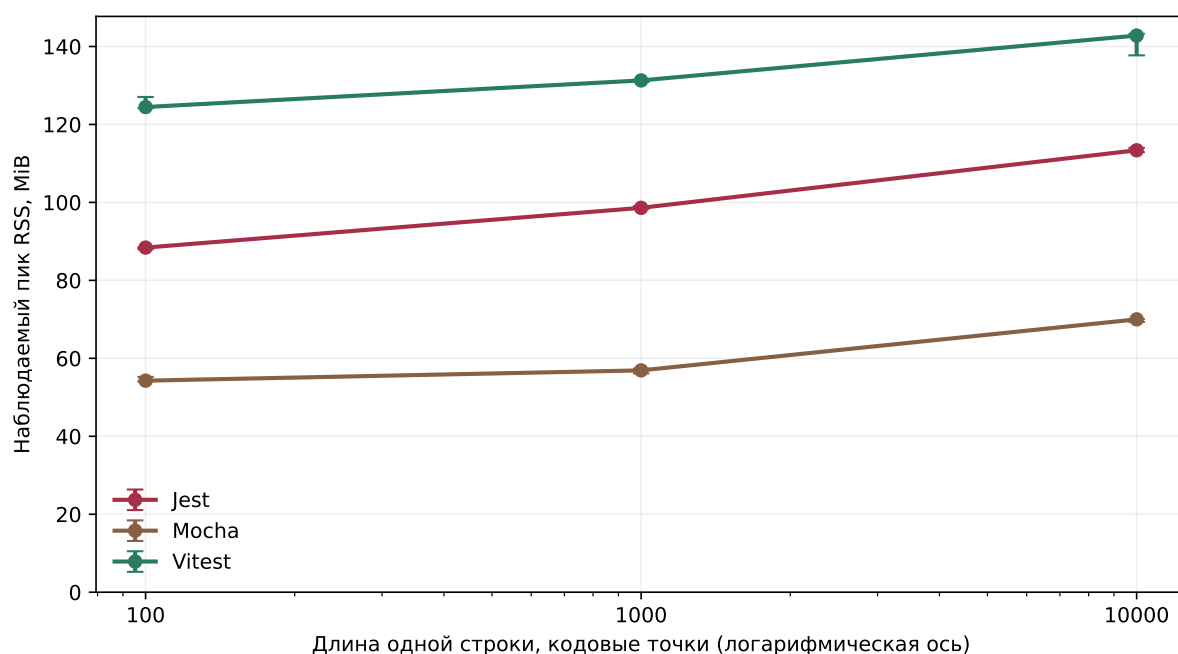


Рис. 3: Наблюдаемый пик RSS, MiB. Точки – медианы, отрезки – Q_1 – Q_3 .

В этой серии Mocha имеет наименьшие медианы времени, CPU-времени и пика RSS на всех трёх размерах. При длине 10000 получены следующие медианы полного времени: Mocha: 483.6 мс, Vitest: 1533.2 мс, Jest: 1758.0 мс. Вывод относится к одному пакетному тесту и данной конфигурации.

Ранжирование по полному времени для каждого размера: 100: Mocha, Jest, Vitest; 1000: Mocha, Jest, Vitest; 10000: Mocha, Vitest, Jest.

Минимальное число отсчётов в основной серии равно 11. Максимальный зарегистрированный промежуток между началами обходов равен 66.6 мс, что больше заданной паузы 10 мс. Это подтверждает необходимость различать настройку паузы и фактическую частоту. Наблюдалось от 1 до 6 процессов на запуск.

Таблица 9: Отдельная серия CSV: 8 строк длины 1000, семь запусков

Раннер	Время, мс	CPU, мс	CPU, %	RSS, MiB
Jest	1142.1	1437.5	123.6	100.9
Mocha	352.1	375.0	106.5	57.6
Vitest	1300.9	1546.9	120.0	131.3

CSV-серия подтверждает работоспособность альтернативного импор-

та под всеми раннерами. Она запускалась отдельно от JSON-серии, поэтому разность времён нельзя полностью приписать формату: между сериями могла измениться внешняя нагрузка. Для причинного сравнения парсеров понадобилась бы отдельная рандомизированная серия.

Проверка эталона, 11 групп проверок встроенным раннером и по 12 тестов в Jest, Mocha и Vitest завершились успешно. Дополнительная приёмка запускает каждый раннер на эталоне, авторском CSV и намеренно неверном ожидании. Последний сценарий должен завершаться ошибкой утверждения; это ожидаемый результат отрицательного теста.

17 Обсуждение и ограничения

Полное время включает постоянные и переменные составляющие. Условная модель

$$T_f(N, q) = A_f + B_f(N, q) + \varepsilon$$

помогает интерпретировать результаты: A_f включает создание процесса и инфраструктуру, B_f – работу с входом, а ε – изменчивость среды. По трём точкам нельзя надёжно оценить универсальную функцию затрат. Если время почти не изменилось при росте небольшого входа, это согласуется с существенной постоянной составляющей, но само по себе не определяет её точный размер.

Низкая средняя CPU-загрузка не обязательно означает меньшее процессорное время. Процесс может долго ожидать файловую систему и иметь малую долю CPU при большом времени ответа. Поэтому $\hat{C}$ и $\hat{U}$ показаны раздельно. При сравнении ресурсов приоритет имеет накопленное время, а процент помогает объяснить соотношение работы и ожидания.

Один worker Vitest не означает один поток операционной системы. Node.js и вспомогательные компоненты могут использовать собственные потоки; часть служб может существовать как дочерний процесс. Ограничивается параллелизм тестовых задач, а не всё внутреннее устройство среды. Сравнение не пытается сделать реализации раннеров идентичными:

различия их инфраструктуры входят в предмет исследования.

Наблюдатель с дискретным опросом не гарантирует фиксацию короткого пика памяти. Отсчёты RSS разных процессов также не являются строго одновременными. Записанный максимум характеризует используемый метод измерения. Для точного учёта завершённых процессов потребовались бы механизмы ОС, например специализированная трассировка и учёт дерева задач, которые выходят за рамки компактного кроссплатформенного стенда.

Наконец, установка и прогрев могут менять состояние кэшей. Измерения выполняются на рабочей Windows-машине, а не на выделенном лабораторном сервере. В отчёте не заявлено, что все фоновые процессы были отключены. Повторный запуск способен дать другие абсолютные величины. Сохранённые версии и входы обеспечивают повторяемую процедуру, но не обещают побитовое совпадение временных показателей.

18 Инструкция по установке и запуску

Для программы требуются Node.js версии не ниже 22 с npm и Python версии не ниже 3.10. В данной апробации использованы конкретные версии, указанные в разделе результатов. Команды выполняются в каталоге `benchmark_app`. Интернет нужен при первой установке пакетов; измеряемые тесты после установки сетевых обращений не требуют.

```
cd benchmark_app
npm ci
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
npm run verify
npm test
npm run demo
npm start -- --text "abba"
npm start -- --file data/input-example.json
npm run test:jest
npm run test:mocha
npm run test:vitest
```

Команды выше рассчитаны на Windows PowerShell. Активация окружения не обязательна: можно непосредственно вызывать его Python. В Linux и macOS вместо `.venv\Scripts \python .exe` используют `.venv/bin/python`. Программа JavaScript не требует отдельной компиляции. Команда `npm ci` устанавливает версии из lock-файла и пригодна для повторяемого code review.

Заготовленные входы включены в проект. Для пересоздания или изменения размера:

```
.venv\Scripts\python.exe generate.py
.venv\Scripts\python.exe generate.py --sizes 200 2000 --cases 12
node src/demo.js data/author-palindromes.csv
```

Увеличение `--cases` повышает число записей, а `--sizes` — длину каждой строки. После изменения нужно заново измерить производительность; старые результаты не описывают новые файлы.

Короткая проверка измерителя:

```
.venv\Scripts\python.exe benchmark.py --input data/size-100.json --runs 2 --output
reports/smoke
```

Основная серия:

```
.venv\Scripts\python.exe benchmark.py --runs 7 --repeats 10 --output reports/new-
json
```

Эквивалентная серия CSV:

```
.venv\Scripts\python.exe benchmark.py --input data/size-1000.csv --runs 7 --output
reports/new-csv
```

Если Node.js не находится в PATH, путь к исполняемому файлу передаётся через `--node`. Параметр `--timeout` задаёт предел в секундах. `--interval-ms` задаёт паузу опроса; изменение этого параметра требует новой серии и должно отражаться в интерпретации результатов. Запись производится в каталог `--output`, поэтому для сохранения прошлой серии выбирают новое имя.

Полная автоматическая приёмка запускается командой `.venv\Scripts \python .exe validate.py`. Она проверяет все три фреймворка, включая на-

меренно неверное ожидание. Сохранённые результаты новой апробации находятся в `reports/2026-09-16-json` и `reports/2026-09-16-csv`. Существующая серия не перезаписывается: при повторе следует выбрать новое имя каталога.

Для сборки документов нужны XeLaTeX и Biber из TeX Live или MiKTeX, пакеты Beamer, biblatex, algorithm и algpseudocode, а также шрифты Times New Roman, Arial и Consolas. Из корня проекта:

```
xelatex -interaction=nonstopmode -halt-on-error My_thesis.tex
biber My_thesis
xelatex -interaction=nonstopmode -halt-on-error My_thesis.tex
xelatex -interaction=nonstopmode -halt-on-error My_thesis.tex
xelatex -interaction=nonstopmode -halt-on-error Presentation.tex
biber Presentation
xelatex -interaction=nonstopmode -halt-on-error Presentation.tex
xelatex -interaction=nonstopmode -halt-on-error Presentation.tex
```

Готовые PDF приложены, поэтому установка LaTeX не нужна для чтения и демонстрации. Для смены персональных данных редактируются титульные блоки двух исходников. При пересчёте экспериментальных результатов сначала обновляются автоматически генерируемые таблицы и диаграммы согласно README.

19 Заключение

В этой серии Mocha имеет наименьшие медианы времени, CPU-времени и пика RSS на всех трёх размерах. При длине 10000 получены следующие медианы полного времени: Mocha: 483.6 мс, Vitest: 1533.2 мс, Jest: 1758.0 мс. Вывод относится к одному пакетному тесту и данной конфигурации.

Разработан компактный стенд сравнения Jest, Mocha и Vitest на общей алгоритмической нагрузке. Он импортирует файлы переменного размера в JSON и CSV, проверяет результат по эталону, выполняет одинаковые утверждения в трёх раннерах и сохраняет данные для воспроизведения эксперимента.

Содержательная проверка предшествует измерению скорости. Независимый перебор коротких строк, специальные случаи, метаморфное свойство и негативные входы проверяют разные классы ошибок. Пошаговый пример показывает связь между радиусами Манакера, найденной подстрокой и количеством палиндромных вхождений.

Сравнение учитывает полное время запуска, накопленное CPU-время и наблюдаемую память дерева процессов. Определения метрик позволяют не смешивать CPU-проценты с процессорным временем, а RSS – с размером JavaScript-кучи. Ограничения дискретного опроса явно включены в методику и трактовку результатов.

Практический результат состоит в проверяемой процедуре выбора для заданной конфигурации. Полученные числа применимы к одному пакетному тесту чистой функции и указанной машине. Следующее развитие исследования – несколько тестовых файлов, режим наблюдения, покрытие и прикладной набор проекта. Эти сценарии требуют отдельных серий, а не распространения текущего ранжирования без измерений.

Список использованных источников

- [1] *Jest CLI Options*. Jest. URL: <https://jestjs.io/docs/30.0/cli> (дата обр. 16.09.2026).
- [2] *Configuring Jest, version 30.0*. Jest. URL: <https://jestjs.io/docs/30.0/configuration> (дата обр. 16.09.2026).
- [3] *Mocha: Command-Line Usage*. Mocha. URL: <https://mochajs.org/running/cli/> (дата обр. 16.09.2026).
- [4] *Configuring Vitest, version 3*. Vitest. URL: <https://v3.vitest.dev/config/> (дата обр. 16.09.2026).
- [5] Giampaolo Rodola. *psutil documentation*. URL: <https://psutil.readthedocs.io/stable/> (дата обр. 16.09.2026).
- [6] *Manacher's Algorithm: Finding all sub-palindromes in $O(N)$* . CP-Algorithms. URL: <https://cp-algorithms.com/string/manacher.html> (дата обр. 16.09.2026).
- [7] *CSV Parse: Sync API*. Adaltas. URL: <https://csv.js.org/parse/api/sync/> (дата обр. 16.09.2026).
- [8] *5. Longest Palindromic Substring*. LeetCode. URL: <https://leetcode.com/problems/longest-palindromic-substring/description/> (дата обр. 16.09.2026).